Lundi 02/11 — Recherche de stage et page sémantique

Kevin Bardoux

lundi 2 novembre 2026

Table des matières

Objectifs 2
Matin — Séance TRE (Jérôme) — lancement de la recherche de stage 3
Au programme 3
Après-midi — TP — « Page sémantique » (HTML & formulaires) . 3
Rappel (5 min) 3
Votre dépôt de l'année : `dwwm-tp` (10 min) 3
Récupérer le kit (5 min) 5
Apport 1 — La structure d'une page (10 min) 5
Structure — attention et vérification 7
Apport 2 — Le HTML sémantique (15 min) 9
Sémantique — attention et vérification 11
Exercice — Le squelette de la page (45 min) 12
Enregistrer (1 min) 12
Apport 3 — Le formulaire (10 min) 12
Formulaire — attention et vérification 13
Exercice — Le formulaire de contact (30 min) 14
Enregistrer (1 min) 15
Vérifier sa page (20 min) 16
Enregistrer et envoyer (2 min) 16
À retenir — cet après-midi 17
Si le temps le permet — Les formulaires, en plus 18
Pour aller plus loin 18
Débrief 18

Sources 18

Note

Compétences visées : C3 · communiquer **Livrable** : le dépôt privé `dwwm-tp` ; `page-perso/index.html` au vert sur `node --test page-perso.test.js` et sans erreur au validateur du W3C.

! Le minimum du jour

- `page-perso/index.html` : le squelette et le formulaire, `node --test page-perso.test.js` au vert.
- Le validateur du W3C n'affiche aucune erreur.
- Le dépôt privé `dwwm-tp` sur GitHub, avec le `.gitignore` et la page du jour envoyée (`git push`).

Le reste (rubriques « Si le temps le permet » et encadrés « Défi ») est la suite naturelle, pour ceux qui avancent.

Objectifs

- Écrire la structure complète d'une page HTML.
- Choisir la balise qui décrit le **rôle** d'un contenu.
- Créer un formulaire accessible : un champ relié à son `label`, un bouton.
- Vérifier sa page avec des tests et le validateur du W3C.

Comment se passe la journée. Le matin, la séance de recherche de stage. L'après-midi commence par votre dépôt de l'année, `dwwm-tp` (une révision de vendredi) ; puis deux notions : la page HTML (sa structure et ses balises sémantiques, en deux apports courts), puis le formulaire. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine l'après-midi.

Matin — Séance TRE (Jérôme) — lancement de la recherche de stage

Au programme

- Séance animée par Jérôme : lancement de la recherche de stage.
- Pas de cours de développement ce matin.
- Cet après-midi, vous commencez votre page personnelle.
- Elle vous servira de portfolio pour vos candidatures.

Après-midi — TP — « Page sémantique » (HTML & formulaires)

Rappel (5 min)

1. Que fait `git commit` ?
2. Que contient une Pull Request ?
3. Que fait une boucle `for` ?

Trois questions courtes, sans regarder vos notes : la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral. Le matin étant consacré à la recherche de stage, le rappel ouvre l'après-midi.

Votre dépôt de l'année : `dwwm-tp` (10 min)

1. GitHub : un dépôt **privé** `dwwm-tp`, vide.
2. `git clone` en SSH dans `~/dwwm`.
3. *Settings*, *Collaborators* : ajoutez le formateur.
4. Premier commit : le `.gitignore` ; puis `git push`.
5. **Réussi si** : `.gitignore` est visible dans `dwwm-tp` sur GitHub.

Niveau : semi-guidé. C'est une révision, en autonomie, de ce que vous avez fait vendredi avec `git-journal` : mêmes gestes, pour le dépôt de toute l'année. À partir d'aujourd'hui, chaque TP s'enregistre dans `dwwm-tp` (un dossier par semaine) et part sur GitHub. Le dépôt est **privé** : seuls vous et le formateur le voyez. Une étape vous échappe ? Les fiches aide-mémoire Git et commit du Drive partagé de la formation, ou la fiche de révision de la semaine 4.

1. Sur GitHub, *New repository* : nom `dwwm-tp`, **Private**, sans README ni `.gitignore`. **Vous devez voir** : la page du dépôt vide, avec son adresse SSH `git@github.com:pseudo/dwwm-tp.git`.
2. Dans Git Bash, en remplaçant `pseudo` par le vôtre :

   ```
   cd ~/dwwm
   git clone git@github.com:pseudo/dwwm-tp.git
   cd dwwm-tp
   ```

   Vous devez voir : `warning: Vous semblez avoir cloné un dépôt vide.` (normal : il est vide), puis l'invite qui se termine par `(main)`.
3. Sur GitHub, dans `dwwm-tp` : *Settings*, *Collaborators*, *Add people* ; tapez le pseudo GitHub du formateur, écrit au tableau. **Vous devez voir** : le formateur dans la liste, avec une invitation en attente.
4. Créez le fichier `.gitignore` et ouvrez le dossier dans VS Code :

   ```
   touch .gitignore
   code .
   ```

   `touch` crée un fichier vide ; `code .` ouvre VS Code sur le dossier courant (le point). Si le terminal répond que la commande `code` est inconnue : ouvrez VS Code, puis *Fichier* > *Ouvrir le dossier...*, et choisissez `dwwm/dwwm-tp`. Dans `.gitignore`, écrivez ces quatre lignes, puis enregistrez :

   ```
   node_modules/
   .env
   .DS_Store
   Thumbs.db
   ```
5. Premier commit et envoi, dans le terminal :

   ```
   git add .gitignore
   git commit -m "Ajoute le .gitignore"
   git push
   ```

Vous devez voir : à la fin, `* [new branch]  main -> main`.

`node_modules/` se réinstalle avec `npm install` ; `.env` contiendra plus tard des mots de passe ; `.DS_Store` (macOS) et `Thumbs.db` (Windows) sont des fichiers du système. Sous macOS, pour que `code .` fonctionne ensuite : dans VS Code, palette de commandes (`Ctrl+Shift+P` (windows), `Cmd+Shift+P` (mac)), *Shell Command: Install "code" command in PATH*.

Récupérer le kit (5 min)

1. Téléchargez l'archive de la semaine.
2. Extrayez-la ; le dossier devient `dwwm/dwwm-tp/semaine-05`.
3. Terminal : `cd ~/dwwm/dwwm-tp/semaine-05`, puis `code .`.
4. Ouvrez `page-perso/index.html` avec **Live Preview**.
5. **Réussi si** : la page s'affiche dans l'aperçu de VS Code.

À l'étape 2 : renommez le dossier extrait `semaine-05`, puis placez-le dans `dwwm/dwwm-tp`. Commande `code` inconnue ? Ouvrez VS Code, puis *Fichier* > *Ouvrir le dossier...*, et choisissez `dwwm/dwwm-tp/semaine-05`. Toute la semaine, ouvrez VS Code sur le dossier `semaine-05` : les tests se lancent depuis ce dossier. Enregistrez le kit dans votre dépôt :

```
git add .
git commit -m "Ajoute le kit de la semaine 5"
```

`git add .` ajoute tout le dossier courant ; le `.gitignore` écarte ce qui ne doit pas partir.

Apport 1 — La structure d'une page (10 min)

- **Pourquoi** : un cadre commun pour afficher la page.
- **Document HTML** : un `head` qui la décrit, un `body` qui l'affiche.

Pourquoi. Sans ces quelques lignes, la page s'affiche mal sur téléphone, les accents peuvent être remplacés par des signes bizarres, et un lecteur d'écran ne sait pas dans quelle langue lire.

Définition. Un document HTML commence par le doctype, puis l'élément `html` qui contient `head` (les informations sur la page) et `body` (le contenu affiché) (MDN Web Docs, s. d.-j). L'attribut `lang`

indique la langue du contenu, utile aux lecteurs d'écran (MDN Web Docs, s. d.-b).

Exemple, commenté ligne par ligne.

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title>Alex — Développeur web en formation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body> … </body>
</html>
```

Ligne 1

`<!doctype html>` : « ce fichier est une page HTML moderne ». Toujours la première ligne.

Ligne 2

`html` contient toute la page ; `lang="fr"` dit la langue du contenu, pour les lecteurs d'écran.

Ligne 3

`head` : les informations **sur** la page, qui ne s'affichent pas dans la page.

Ligne 4

`charset="utf-8"` : les accents s'affichent correctement.

Ligne 5

`viewport` : la page s'adapte à la largeur du téléphone.

Ligne 6

`title` : le texte de l'onglet, et le titre dans les moteurs de recherche.

Ligne 7

`link` relie la feuille de style `style.css`, dans le même dossier.

Ligne 9

`body` : tout ce qui s'affiche dans la page.

Pas à pas. Le formateur lit le code ligne par ligne, avec les annotations ci-dessus ; vous retrouvez chaque ligne dans `page-perso/index.html`.

Structure — attention et vérification

> ⚠ Attention
>
> Oublier la ligne `viewport` : sur téléphone, la page s'affiche en tout petit, comme sur un écran d'ordinateur réduit. Elle se place dans le `head`.

Avant / après : la même page, affichée par un téléphone de 375 px, sans puis avec la ligne `viewport`.



Figure 1. – Sans viewport : tout est minuscule

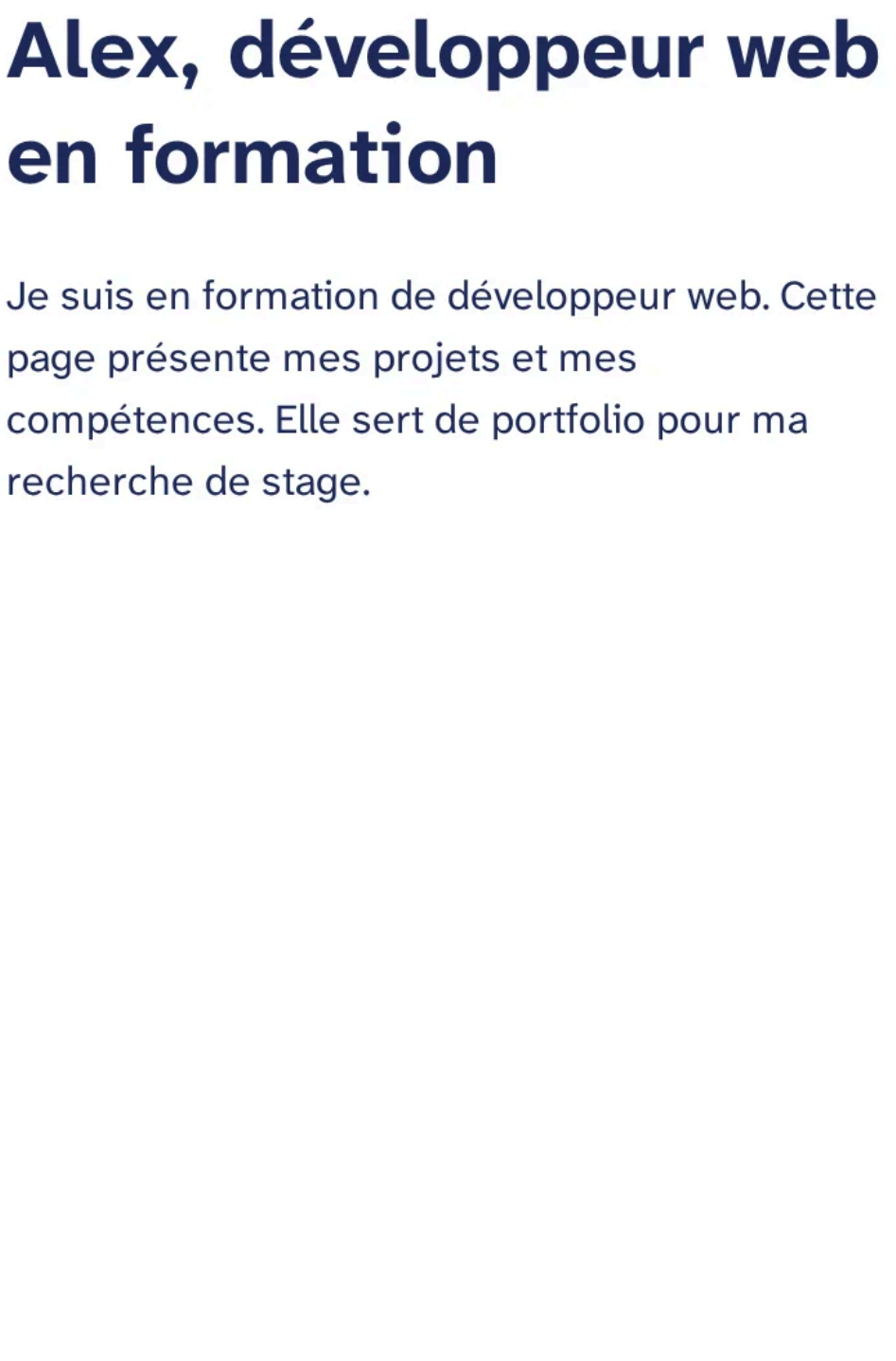


Figure 2. – Avec viewport : la page est lisible

- **Vérifiez** : où s'affiche le contenu de `title` ?

Apport 2 — Le HTML sémantique (15 min)

- **Pourquoi** : lecteurs d'écran et moteurs lisent les balises.
- **Sémantique** : la balise dit le **rôle**, pas l'apparence.

Pourquoi. Les lecteurs d'écran s'en servent pour naviguer dans la page (aller au menu, au contenu principal) ; les moteurs de

recherche comprennent mieux votre contenu ; le code est plus lisible pour vous et vos collègues. Un HTML sémantique est la première base de l'accessibilité (MDN Web Docs, s. d.-g).

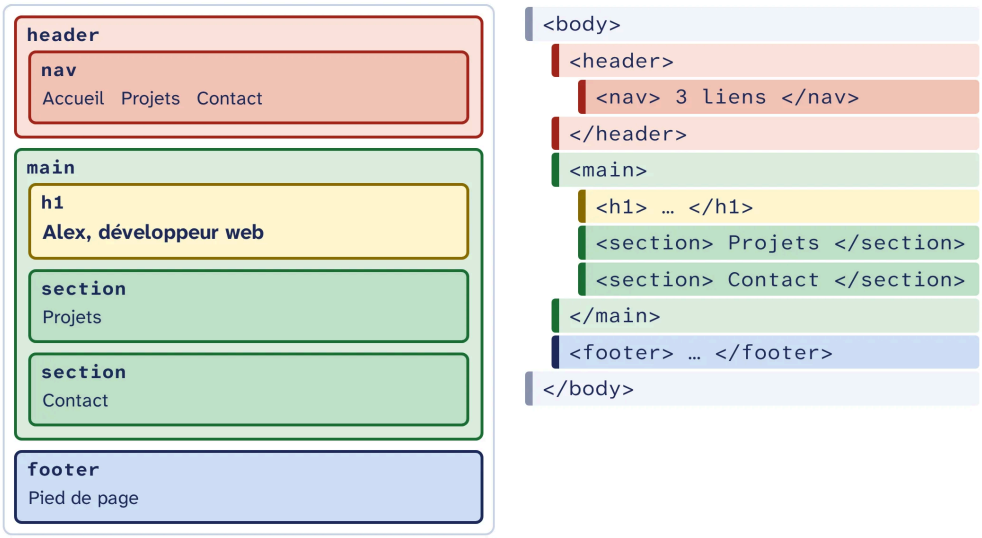
Définition. La sémantique, c'est le sens d'un élément : un `h1` est un titre, un `nav` une navigation (MDN Web Docs, s. d.-h). Les éléments de structure (`header`, `nav`, `main`, `section`, `article`, `footer`) sont décrits dans « Structurer des documents » (MDN Web Docs, s. d.-i). Les titres se suivent dans l'ordre (`h1`, puis `h2`, puis `h3`), un seul `h1` par page ; l'attribut `alt` d'une image donne son équivalent textuel, lu par les lecteurs d'écran (MDN Web Docs, s. d.-c).



Figure 3. – Les zones d'une page : en haut `header` et son `nav`, au centre `main` avec le `h1` et une `section` par partie, en bas `footer`.

La page et son arbre HTML. À gauche, une vraie page, chaque zone colorée ; à droite, son code, chaque balise de la même couleur que sa zone :

La page et son arbre HTML : chaque zone a la même couleur des deux côtés



Exemple. Le même plan, en HTML :

```
<header>
  <nav> … </nav>
</header>
<main>
  <h1>Alex, développeur web en formation</h1>
  <section id="projets"> <h2>Projets</h2> … </section>
</main>
<footer> … </footer>
```

Pas à pas. Le formateur dessine la page au tableau en quatre zones ; vous les nommez avec lui.

1. En haut : l'en-tête, `header`, avec le menu, `nav`.
2. Au centre : le contenu principal, `main`, qui commence par le `h1`.
3. Dans `main` : une `section` par partie, chacune avec son `h2`.
4. En bas : le pied de page, `footer`.

Sémantique — attention et vérification

> ⚠ Attention
>
> Plusieurs `h1`, ou des titres qui sautent un niveau (`h2` puis `h4`) : le lecteur d'écran perd le plan de la page. Un seul `h1`, puis les niveaux dans l'ordre.

- **Vérifiez** : quelle balise pour un menu de liens ?

Exercice — Le squelette de la page (45 min)

1. Ensemble (5 min) : le `head` complet.
2. Seuls : le `body`, en-tête avec navigation, puis `main`.
3. Quatre sections : présentation, compétences, projets, contact.
4. Prénom **ou** pseudonyme : aucune donnée personnelle obligatoire.
5. **Réussi si** : `page-perso.test.js` passe (sauf le formulaire).

Niveau : guidé.

Au début, plusieurs tests restent rouges : ceux du formulaire passeront après l'apport 3. Le fichier `layout.test.js` concerne la mise en page de jeudi : ignorez-le d'ici là (lancez seulement `node --test page-perso.test.js`). Pour les projets, citez ce que vous avez déjà réalisé : problem sets, projet Scratch, Git Journal.

Enregistrer (1 min)
Dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add page-perso/index.html
git commit -m "Ajoute la structure de la page personnelle"
```

Réussi si : `git status` affiche `rien à valider, la copie de travail est propre`.

Apport 3 — Le formulaire (10 min)

- **Pourquoi** : recevoir une information du visiteur.
- **Formulaire** : des champs, chacun relié à son `label`.

Pourquoi. Une page de contact, une recherche, une inscription : le visiteur écrit, le site reçoit. Un formulaire mal construit est inutilisable au clavier ou avec un lecteur d'écran.

Définition. Un formulaire (`form`) regroupe des champs de saisie (`input`) et un bouton d'envoi (`button`). Un `label` associé à son champ par `for` et `id` rend le formulaire accessible, et agrandit la

zone cliquable (MDN Web Docs, s. d.-e). Les autres types de champs et l'attribut `required` sont dans le Défi de l'exercice.

Exemple.

```
<form>
  <label for="nom">Votre nom</label>
  <input type="text" id="nom" name="nom">

  <button type="submit">Envoyer</button>
</form>
```

Résultat dans le navigateur, sans feuille de style (cliquez sur une étiquette : le curseur va dans son champ) :



Pas à pas. Le formateur écrit le premier champ ; vous le recopiez.

1. Écrivez le `label`, avec `for="nom"`.
2. Écrivez l'`input`, avec le même `id="nom"`.
3. Cliquez sur le texte du `label` dans l'aperçu : le curseur va dans le champ. Le lien fonctionne.
4. Écrivez le `button`, avec `type="submit"`.

Formulaire — attention et vérification

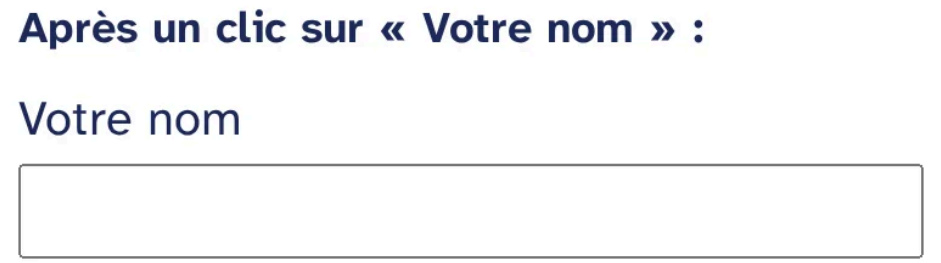
> ⚠ Attention
>
> Un `for` qui ne correspond à aucun `id` : le champ n'a plus d'étiquette pour le lecteur d'écran. Les deux valeurs doivent être identiques, au caractère près.

Avant / après : le même clic sur « Votre nom ». Quand `for` et `id` sont identiques, le curseur va dans le champ (il est entouré) ; sinon, rien ne se passe.




```
<label for="nom">
<input id="nom">
```

Figure 4. – for=« nom » et id=« nom » : le champ est relié




```
<label for="name">
<input id="nom">
```

Figure 5. – for=« name » et id=« nom » : le lien est cassé

- **Vérifiez** : cliquez sur un `label` : où va le curseur ?

Exercice — Le formulaire de contact (30 min)

1. Seuls : un champ « Votre nom », relié à son `label` (`for` = `id`).
2. Un bouton « Envoyer ».
3. Ajoutez : « Formulaire non fonctionnel (démonstration) ».
4. **Réussi si** : les tests du formulaire passent.

> **Défi**
>
> Le formulaire complet : un champ d'adresse e-mail (`type="email"`), un message (`textarea`), chacun avec son `label`, et `required` sur les trois. **Réussi si** : `node --test page-perso.defi.test.js` passe.

Niveau : guidé.

Pour le Défi : `type="email"` affiche le bon clavier sur téléphone et vérifie la forme de l'adresse ; il existe d'autres types (`number`, `date`…) (MDN Web Docs, s. d.-d). `textarea` accepte un texte sur plusieurs lignes ; `required` empêche l'envoi si le champ est vide (MDN Web Docs, s. d.-a). Pour aller plus loin : « Formulaires et boutons en HTML » (MDN Web Docs, s. d.-f).

Aucune adresse e-mail en clair dans le code : un test le vérifie. Le formulaire est une **maquette** : GitHub Pages ne fait tourner aucun serveur, il ne peut donc rien envoyer. Annoncez-le clairement au visiteur, par exemple : « Formulaire non fonctionnel : c'est une démonstration, il n'envoie rien. » Pour être contacté, un lien vers votre profil GitHub suffit.

Enregistrer (1 min)
Dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add page-perso/index.html
git commit -m "Ajoute le formulaire de contact"
```

Réussi si : `git status` affiche `rien à valider, la copie de travail est propre`.

Vérifier sa page (20 min)

Vérifier avant de rendre

1. **Ouvrir la page** : Live Preview, puis *Ouvrir dans le navigateur*.
2. **Console** (F12, onglet *Console*) : aucune ligne rouge.
3. **VS Code** : aucun souligné ondulé ; survolez-le pour lire le message.

Une page qui « marche » dans l'aperçu peut cacher une erreur. Une ligne rouge dans la console signale par exemple une feuille de style ou une image introuvable (`404`, *Failed to load resource*) : vérifiez le nom et le chemin du fichier, au caractère près. Un souligné ondulé dans VS Code (rouge ou jaune) marque une erreur de syntaxe CSS ou une propriété inconnue ; le panneau *Problèmes* (menu *Affichage*) les liste toutes. VS Code ne signale pas toutes les erreurs de HTML : c'est le rôle du validateur du W3C, ci-dessous. Faites ces trois vérifications avant chaque rendu, toute la semaine.

1. `node --test page-perso.test.js` : tous les tests au vert.
2. Validateur du W3C : *Validate by Direct Input*, collez votre code.
3. Corrigez chaque erreur, en commençant par la première.
4. **Réussi si** : aucune erreur au validateur.

Niveau : guidé.

Le validateur du W3C vérifie que le HTML respecte la norme (W3C, s. d.). Les tests du kit vérifient d'autres points (sémantique, accessibilité du formulaire, données personnelles) : les deux sont complémentaires. Au palier 1, vous pouvez demander à CS50.ai d'expliquer un message du validateur, pas de corriger le code à votre place.

Enregistrer et envoyer (2 min)

Fin du TP : dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-05` :

```
git status
git add page-perso/index.html
```

```
git commit -m "Corrige les erreurs du validateur"
git push
```

Réussi si : `git status` affiche `Votre branche est à jour avec 'origin/main'` et `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la règle de la semaine 4 : ce que fait le commit, à l'impératif, en une ligne (fiche aide-mémoire commit du Drive partagé, ou fiche de révision).

À retenir — cet après-midi

> À retenir
>
> - **Structure** : `lang`, `charset`, `viewport`, `title` dans le `head`.
> - **Sémantique** : la balise dit le rôle ; un seul `h1`, titres dans l'ordre.
> - **Formulaire** : chaque champ a son `label` (`for` = `id`) ; un bouton d'envoi.

Le détail, notion par notion :

- **Structure** : doctype, `html`, `head` (ce qui décrit la page), `body` (ce qui s'affiche) ; sans `viewport`, la page est minuscule sur téléphone.
- **Sémantique** : `header`, `nav`, `main`, `section`, `footer` ; un seul `h1`, puis `h2`, `h3` ; un `alt` sur chaque image.
- **Formulaire** : `label` relié par `for` et `id`, un `button` ; dans le Défi, `type` adapte le clavier et `required` rend le champ obligatoire.

Critères de réussite

- ☐ `node --test page-perso.test.js` : tous les tests au vert
- ☐ Le validateur du W3C n'affiche aucune erreur
- ☐ Le formulaire annonce qu'il n'est pas encore fonctionnel

Si le temps le permet — Les formulaires, en plus

> Défi
>
> - `fieldset` et `legend` pour regrouper des champs.
> - `autocomplete="email"` pour aider le navigateur à préremplir.
> - `placeholder` ne remplace jamais un `label`.

Pour aller plus loin

- Ajoutez une balise `meta name="description"`.
- Testez votre page au clavier seul, avec la touche Tab.
- Lecture de référence : *Learn HTML*, sur web.dev.

Learn HTML, le parcours de l'équipe Chrome sur web.dev, reprend tout le HTML chapitre par chapitre, de la structure aux formulaires (web.dev (Google Chrome), s. d.). La traduction française est automatique : en cas de doute, la version anglaise (bouton de langue en haut de la page) fait foi. Lecture libre, sans temps obligatoire.

Débrief

- Pourquoi un seul `h1` par page ?
- À quoi sert le lien entre `for` et `id` ?
- Quelle erreur du validateur avez-vous corrigée en premier ?

Sources

MDN Web Docs. s. d.-a. « Attribut HTML : required ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Attributes/required.

MDN Web Docs. s. d.-b. « Attribut HTML universel : lang ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Global_attributes/lang.

MDN Web Docs. s. d.-c. « Élément HTML <img> : l'élément d'image embarquée ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Elements/img.

MDN Web Docs. s. d.-d. « Élément HTML <input> : l'élément de saisie dans un formulaire ». Consulté le 26 septembre 2026.

https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Elements/input.

MDN Web Docs. s. d.-e. « Élément HTML <label> : l'élément de libellé ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Elements/label.

MDN Web Docs. s. d.-f. « Formulaires et boutons en HTML - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Structuring_content/HTML_forms.

MDN Web Docs. s. d.-g. « HTML : une bonne base pour l'accessibilité - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Accessibility/HTML.

MDN Web Docs. s. d.-h. « Sémantique - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/Semantics.

MDN Web Docs. s. d.-i. « Structurer des documents - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Structuring_content/Structuring_documents.

MDN Web Docs. s. d.-j. « Syntaxe de base du HTML - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Core/Structuring_content/Basic_HTML_syntax.

W3C. s. d. « The W3C Markup Validation Service ». Consulté le 26 septembre 2026. https://validator.w3.org/.

web.dev (Google Chrome). s. d. « Learn HTML (traduction automatique en français ; textes sous licence CC BY 4.0) ». Consulté le 2 octobre 2026. https://web.dev/learn/html/welcome?hl=fr.